Credit Card Fraud Detection using Neural Network (GPU Accelerated)

April 7, 2026

1 Introduction

Fraud detection in financial transactions is a critical application of machine learning, especially due to the highly imbalanced nature of real-world datasets. This project focuses on building a robust classification model to detect fraudulent credit card transactions using a neural network trained on GPU-accelerated infrastructure.

The dataset consists of anonymized transaction features and a binary target variable indicating fraud (1) or non-fraud (0). The challenge lies in detecting rare fraud cases among a vast majority of legitimate transactions.

[1]:
```
!module load cuda/12.6
!module load cudnn/8.9.7.29-12-cuda12.6
import torch
print("CUDA version:", torch.version.cuda)
print("cuDNN version:", torch.backends.cudnn.version())
print("GPU available?", torch.cuda.is_available())
print("Device count:", torch.cuda.device_count())
print("Device name:", torch.cuda.get_device_name(0))
import torch.nn as nn
import pandas as pd
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import random_split
import numpy as np
```

```
Loading cuda version 12.6
Loading cuda version 12.6
Loading cudnn version 8.9.7.29-12
CUDA version: 12.6
cuDNN version: 91002
GPU available? True
Device count: 1
Device name: NVIDIA A16
```

2 Dataset Description

The dataset contains 284,807 transactions, with 31 features, including:

1) 28 anonymized numerical features (V1–V28)
2) Time and Amount
3) Target variable: Class

Key Observation:

1) Fraud cases: 492 (~0.17%)
2) Non-fraud cases: 284,315 (~99.83%)

This severe imbalance makes accuracy alone an unreliable metric.

[2]:
```
data = pd.read_csv("creditcard.csv")
```

[4]:
```
data.shape[0]
```

[4]:
```
284807
```

[5]:
```
data.head()
```

[5]:
```
   Time        V1        V2        V3        V4        V5        V6        V7  \
0   0.0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388  0.239599
1   0.0  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361 -0.078803
2   1.0 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499  0.791461
3   1.0 -0.966272 -0.185226  1.792993 -0.863291 -0.010309  1.247203  0.237609
4   2.0 -1.158233  0.877737  1.548718  0.403034 -0.407193  0.095921  0.592941

         V8        V9  ...       V21       V22       V23       V24       V25  \
0  0.098698  0.363787  ... -0.018307  0.277838 -0.110474  0.066928  0.128539
1  0.085102 -0.255425  ... -0.225775 -0.638672  0.101288 -0.339846  0.167170
2  0.247676 -1.514654  ...  0.247998  0.771679  0.909412 -0.689281 -0.327642
3  0.377436 -1.387024  ... -0.108300  0.005274 -0.190321 -1.175575  0.647376
4 -0.270533  0.817739  ... -0.009431  0.798278 -0.137458  0.141267 -0.206010

        V26       V27       V28  Amount  Class
0 -0.189115  0.133558 -0.021053  149.62      0
1  0.125895 -0.008983  0.014724    2.69      0
2 -0.139097 -0.055353 -0.059752  378.66      0
3 -0.221929  0.062723  0.061458  123.50      0
4  0.502292  0.219422  0.215153   69.99      0

[5 rows x 31 columns]
```

[34]:
```
#mask = (
#    (data['Class'] == 1)
#)
```

```
#filtered_data = data[mask]
```

[6]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

3 Data Analysis

Class Imbalance

1) Pie chart and count plot clearly show extreme imbalance.
2) Requires special handling during training.

Correlation Analysis

1) Features like V11, V4, V2 show positive correlation with fraud.
2) Features like V14, V17, V12 show strong negative correlation.

Data Quality

1) No missing values in the dataset.
2) All features are numerical → suitable for neural networks.

[7]:
```
data.describe()
```

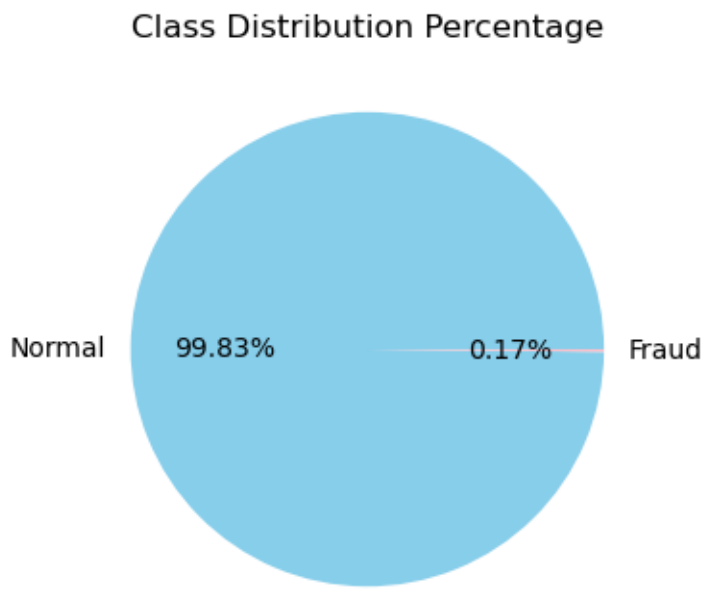
[7]:
```
                Time            V1            V2            V3            V4  \
count  284807.000000  2.848070e+05  2.848070e+05  2.848070e+05  2.848070e+05
mean    94813.859575  1.168375e-15  3.416908e-16 -1.379537e-15  2.074095e-15
std     47488.145955  1.958696e+00  1.651309e+00  1.516255e+00  1.415869e+00
min         0.000000 -5.640751e+01 -7.271573e+01 -4.832559e+01 -5.683171e+00
25%     54201.500000 -9.203734e-01 -5.985499e-01 -8.903648e-01 -8.486401e-01
50%     84692.000000  1.810880e-02  6.548556e-02  1.798463e-01 -1.984653e-02
75%    139320.500000  1.315642e+00  8.037239e-01  1.027196e+00  7.433413e-01
max    172792.000000  2.454930e+00  2.205773e+01  9.382558e+00  1.687534e+01

                 V5            V6            V7            V8            V9  \
count  2.848070e+05  2.848070e+05  2.848070e+05  2.848070e+05  2.848070e+05
mean   9.604066e-16  1.487313e-15 -5.556467e-16  1.213481e-16 -2.406331e-15
std    1.380247e+00  1.332271e+00  1.237094e+00  1.194353e+00  1.098632e+00
min   -1.137433e+02 -2.616051e+01 -4.355724e+01 -7.321672e+01 -1.343407e+01
25%   -6.915971e-01 -7.682956e-01 -5.540759e-01 -2.086297e-01 -6.430976e-01
50%   -5.433583e-02 -2.741871e-01  4.010308e-02  2.235804e-02 -5.142873e-02
75%    6.119264e-01  3.985649e-01  5.704361e-01  3.273459e-01  5.971390e-01
max    3.480167e+01  7.330163e+01  1.205895e+02  2.000721e+01  1.559499e+01

       ...           V21           V22           V23           V24  \
count  ...  2.848070e+05  2.848070e+05  2.848070e+05  2.848070e+05
mean   ...  1.654067e-16 -3.568593e-16  2.578648e-16  4.473266e-15
std    ...  7.345240e-01  7.257016e-01  6.244603e-01  6.056471e-01
min    ... -3.483038e+01 -1.093314e+01 -4.480774e+01 -2.836627e+00
25%    ... -2.283949e-01 -5.423504e-01 -1.618463e-01 -3.545861e-01
50%    ... -2.945017e-02  6.781943e-03 -1.119293e-02  4.097606e-02
75%    ...  1.863772e-01  5.285536e-01  1.476421e-01  4.395266e-01
max    ...  2.720284e+01  1.050309e+01  2.252841e+01  4.584549e+00

                V25           V26           V27           V28         Amount  \
count  2.848070e+05  2.848070e+05  2.848070e+05  2.848070e+05  284807.000000
mean   5.340915e-16  1.683437e-15 -3.660091e-16 -1.227390e-16      88.349619
```

```
std    5.212781e-01  4.822270e-01  4.036325e-01  3.300833e-01     250.120109
min   -1.029540e+01 -2.604551e+00 -2.256568e+01 -1.543008e+01       0.000000
25%   -3.171451e-01 -3.269839e-01 -7.083953e-02 -5.295979e-02       5.600000
50%    1.659350e-02 -5.213911e-02  1.342146e-03  1.124383e-02      22.000000
75%    3.507156e-01  2.409522e-01  9.104512e-02  7.827995e-02      77.165000
max    7.519589e+00  3.517346e+00  3.161220e+01  3.384781e+01   25691.160000

               Class
count  284807.000000
mean        0.001727
std         0.041527
min         0.000000
25%         0.000000
50%         0.000000
75%         0.000000
max         1.000000

[8 rows x 31 columns]
```

[8]:
```
data["Class"].value_counts()
```

[8]:
```
Class
0    284315
1       492
Name: count, dtype: int64
```

[9]:
```
import matplotlib.pyplot as plt
plt.figure(figsize= (6,4))
class_count = data["Class"].value_counts()
labels = ["Normal", "Fraud"]

plt.pie(class_count, labels = labels, autopct = "%1.2f%%", colors = ["skyblue",␣
 ↪"pink"])
plt.title("Class Distribution Percentage")
plt.show()
```



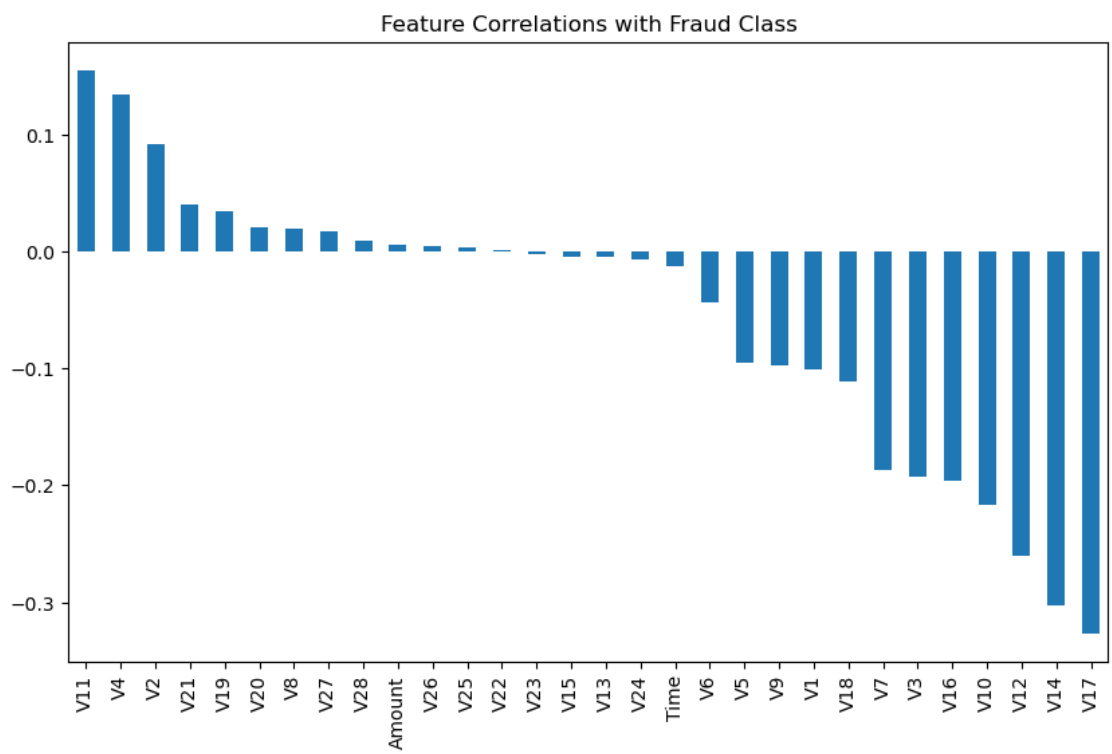
[10]:
```
import seaborn as sns
plt.figure(figsize=(6,4))
sns.countplot(x='Class', data=data)
plt.title("Transaction Count by Class")
#plt.xticks([0,1], ['Normal', 'Fraud'])
plt.show()
```

[11]:
```
corr = data.corr()["Class"].sort_values(ascending=False)
print("Top Postive Correlations: \n", corr.head(10))
print("\nTop Negative Correlations: \n", corr.tail(10))
```

```
Top Postive Correlations:
 Class    1.000000
V11      0.154876
V4       0.133447
V2       0.091289
V21      0.040413
V19      0.034783
V20      0.020090
V8       0.019875
V27      0.017580
V28      0.009536
Name: Class, dtype: float64

Top Negative Correlations:
 V9    -0.097733
V1    -0.101347
V18   -0.111485
V7    -0.187257
V3    -0.192961
V16   -0.196539
```

```
V10   -0.216883
V12   -0.260593
V14   -0.302544
V17   -0.326481
Name: Class, dtype: float64
```

[12]:
```
plt.figure(figsize = (10, 6))
corr.drop("Class").plot(kind = "bar")
plt.title("Feature Correlations with Fraud Class")
plt.show()
```



[14]:
```
data.isna().sum().sort_values(ascending=False)
```

[14]:
```
Time      0
V1        0
V2        0
V3        0
V4        0
V5        0
V6        0
V7        0
V8        0
V9        0
```

```
V10       0
V11       0
V12       0
V13       0
V14       0
V15       0
V16       0
V17       0
V18       0
V19       0
V20       0
V21       0
V22       0
V23       0
V24       0
V25       0
V26       0
V27       0
V28       0
Amount    0
Class     0
dtype: int64
```

[ ]:
```
data = data.drop(columns = ["Time", "Amount"])
```

[19]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 29 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   V1      284807 non-null  float64
 1   V2      284807 non-null  float64
 2   V3      284807 non-null  float64
 3   V4      284807 non-null  float64
 4   V5      284807 non-null  float64
 5   V6      284807 non-null  float64
 6   V7      284807 non-null  float64
 7   V8      284807 non-null  float64
 8   V9      284807 non-null  float64
 9   V10     284807 non-null  float64
 10  V11     284807 non-null  float64
 11  V12     284807 non-null  float64
 12  V13     284807 non-null  float64
 13  V14     284807 non-null  float64
 14  V15     284807 non-null  float64
 15  V16     284807 non-null  float64
```

```
 16  V17     284807 non-null  float64
 17  V18     284807 non-null  float64
 18  V19     284807 non-null  float64
 19  V20     284807 non-null  float64
 20  V21     284807 non-null  float64
 21  V22     284807 non-null  float64
 22  V23     284807 non-null  float64
 23  V24     284807 non-null  float64
 24  V25     284807 non-null  float64
 25  V26     284807 non-null  float64
 26  V27     284807 non-null  float64
 27  V28     284807 non-null  float64
 28  Class   284807 non-null  int64
dtypes: float64(28), int64(1)
memory usage: 63.0 MB
```

[20]:
```
conv_data = torch.tensor(
    data.values,
    dtype=torch.float32
)
```

4 Data Preprocessing

Dropped unnecessary features (Time, Amount)

Converted dataset into PyTorch tensors

Split into:

1) Training: 80%
2) Validation: 10%
3) Testing: 10%

[21]:
```
X = conv_data[:, :-1]
#y = data[:, -3:].unsqueeze(1)
y = conv_data[:, -1]

dataset = TensorDataset(X, y)
```

[22]:
```
def normalize(train):
    mean = train.mean(dim=0, keepdim=True)
    std  = train.std(dim=0, keepdim=True)
    train = (train - mean) / (std + 1e-8)
    return train
```

[41]:
```
N = len(dataset)
n_train = int(0.8 * N)
n_val   = int(0.1 * N)
```

```
n_test  = N - n_train - n_val

train_ds, val_ds, test_ds = random_split(
    dataset,
    [n_train, n_val, n_test],
    generator=torch.Generator().manual_seed(42)  # reproducible
)
```

5 Model Architecture

A Multi-Layer Perceptron (MLP) was used:

1) Input Layer: 28 features
2) Hidden Layer 1: 128 neurons (ReLU)
3) Hidden Layer 2: 128 neurons (ReLU)
4) Output Layer: 1 neuron (Binary classification)

[42]:
```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(28, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        return self.net(x)
```

6 Handling Class Imbalance

To address imbalance, the loss function used:

1) Binary Cross Entropy with Logits (BCEWithLogitsLoss)
2) Positive class weighting: $pos_{weight} = \frac{N_{negative}}{N_{positive}}$

This ensures fraud cases are penalized more during training.

[63]:
```
model = MLP()
#criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=1e-4)
pos_weight = torch.tensor([284315 / 492]).to(device)  # weight = N_neg / N_pos
criterion = torch.nn.BCEWithLogitsLoss(pos_weight=pos_weight)
```

7 Training Strategy

1) Optimizer: Adam (learning rate = 1e-4)
2) Batch size: 1024
3) Epochs: 500
4) Early stopping implemented
5) GPU acceleration enabled (NVIDIA A16)

Training was optimized using:

1) torch.compile() for performance
2) Multi-worker DataLoaders
3) GPU memory pinning

[64]:
```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

#  Compile the model (PyTorch 2.0+)
model = torch.compile(model, backend="inductor")  # or backend="nvfuser" on GPU

#  DataLoaders with multiple workers
train_loader = DataLoader(train_ds, batch_size=1024, shuffle=True,␣
 ↪num_workers=4, pin_memory=True)
val_loader   = DataLoader(val_ds, batch_size=1024, shuffle=False, num_workers=4,␣
 ↪pin_memory=True)
test_loader = DataLoader(test_ds, batch_size=1024)
```

[65]:
```
epochs = 500
patience = 500
best_val = float('inf')
wait = 0
best_model_state = None

for epoch in range(epochs):
    #  Training loop
    model.train()
    for xb, yb in train_loader:
        xb, yb = xb.to(device, non_blocking=True), yb.to(device,␣
 ↪non_blocking=True)
        #yb = yb.squeeze(1)
        yb = yb.float().unsqueeze(1)
        optimizer.zero_grad()
        pred = model(xb)
        loss = criterion(pred, yb)
        loss.backward()
        optimizer.step()

    #  Validation every 100 epochs
```

```
    if epoch % 100 == 0:
        model.eval()
        val_loss = 0.0
        with torch.no_grad():
            for xb, yb in val_loader:
                xb, yb = xb.to(device, non_blocking=True), yb.to(device,␣
↪non_blocking=True)
                #yb = yb.squeeze(1)
                yb = yb.float().unsqueeze(1)
                val_loss = criterion(model(xb), yb).item()

        print(f"Epoch {epoch}: Train Loss = {loss:.4e} Val Loss = {val_loss:.
↪4e}")

        #  Early stopping
        if val_loss < best_val:
            best_val = val_loss
            best_model_state = model.state_dict()
            torch.save(model.state_dict(), "best_model.pth")
            wait = 0
        else:
            wait += 100

        if wait >= patience:
            print("Early stopping triggered")
            break

#  Restore best model
if best_model_state is not None:
    model.load_state_dict(best_model_state)
```

```
Epoch 0: Train Loss = 3.6442e-01 Val Loss = 3.6943e-01
Epoch 100: Train Loss = 1.1662e-02 Val Loss = 3.0416e-02
Epoch 200: Train Loss = 6.3164e-03 Val Loss = 1.4823e-02
Epoch 300: Train Loss = 2.7126e-03 Val Loss = 3.7566e-03
Epoch 400: Train Loss = 4.5507e-04 Val Loss = 1.0646e-02
```

[69]:
```
from sklearn.metrics import (
    precision_score, recall_score, f1_score,
    roc_auc_score, average_precision_score,
    confusion_matrix, classification_report
)

y_true = []
y_pred = []
y_proba = []
```

```python
model.eval()

with torch.no_grad():
    for xb, yb in test_loader:

        xb = xb.to(device)
        outputs = model(xb)

        probs = torch.sigmoid(outputs)       # probability
        preds = (probs > 0.5).int()          # predicted class

        y_true.extend(yb.cpu().numpy())
        y_pred.extend(preds.cpu().numpy())
        y_proba.extend(probs.cpu().numpy())

# convert to arrays
y_true = np.array(y_true).squeeze()
y_pred = np.array(y_pred).squeeze()
y_proba = np.array(y_proba).squeeze()

# ---- METRICS ----
precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
roc_auc = roc_auc_score(y_true, y_proba)
pr_auc = average_precision_score(y_true, y_proba)

cm = confusion_matrix(y_true, y_pred)
report = classification_report(y_true, y_pred)

print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-score: {f1:.4f}")
print(f"ROC-AUC: {roc_auc:.4f}")
print(f"PR-AUC: {pr_auc:.4f}")

print("\nConfusion Matrix")
print(cm)

print("\nClassification Report")
print(report)
```

```
Precision: 0.8167
Recall: 0.8448
F1-score: 0.8305
ROC-AUC: 0.9538
PR-AUC: 0.7993
```

```
Confusion Matrix
[[28413    11]
 [    9    49]]

Classification Report
              precision    recall  f1-score   support

         0.0       1.00      1.00      1.00     28424
         1.0       0.82      0.84      0.83        58

    accuracy                           1.00     28482
   macro avg       0.91      0.92      0.92     28482
weighted avg       1.00      1.00      1.00     28482
```

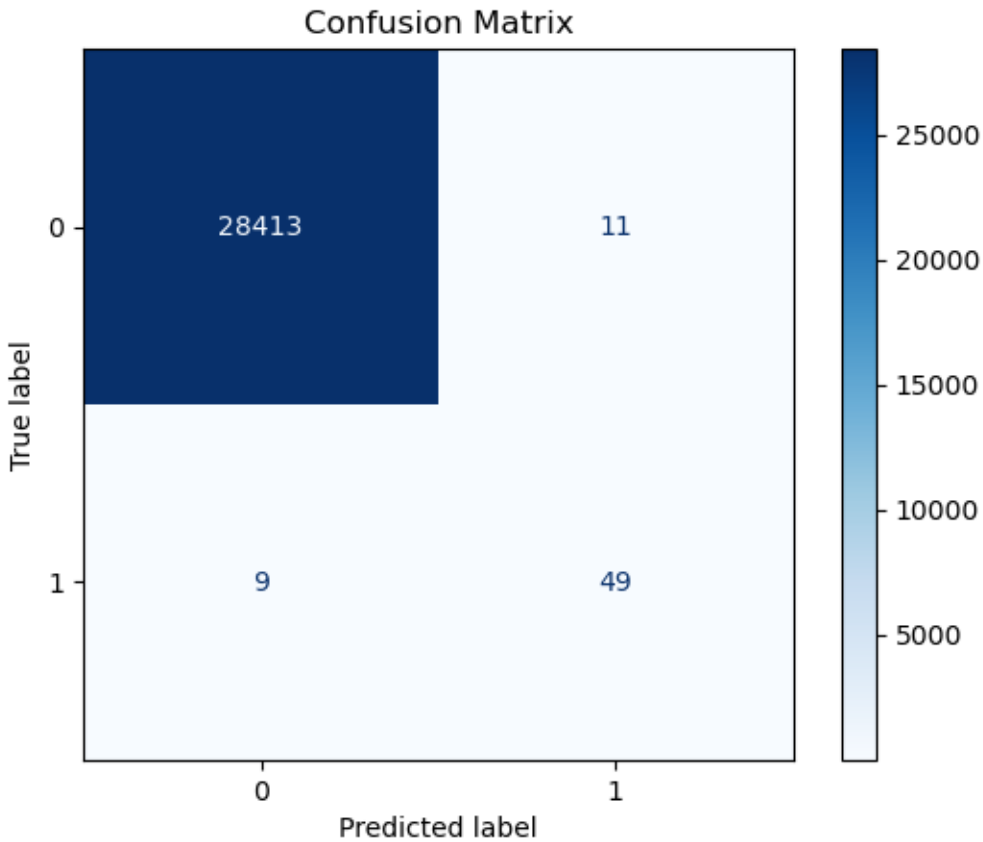
8 Visualization Results

ROC curve (page 15–16) shows excellent separability (AUC 0.95)

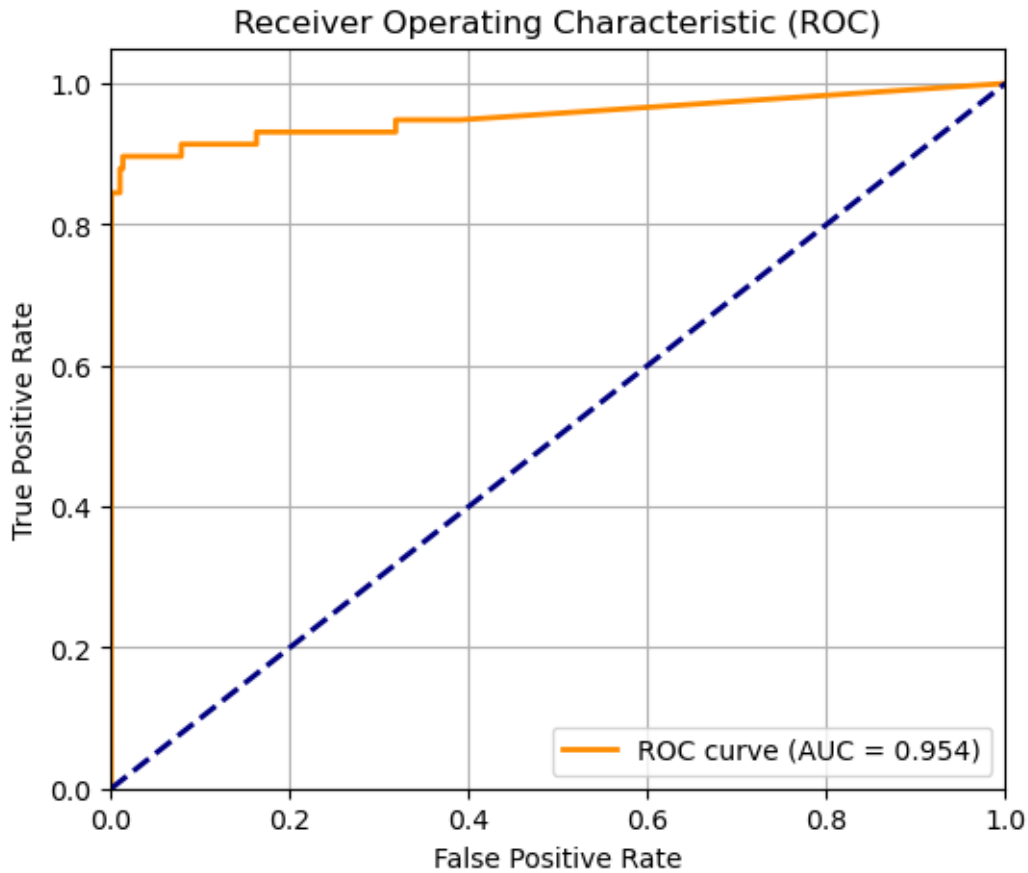
Confusion matrix confirms balanced performance across classes

1) Very low false positives
2) Strong fraud detection capability

[70]:
```python
from sklearn.metrics import ConfusionMatrixDisplay

# Option 1: Using sklearn's built-in display
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[0, 1])
disp.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix")
plt.show()
```



[68]:
```
from sklearn.metrics import roc_curve, auc

# Compute ROC curve
fpr, tpr, thresholds = roc_curve(y_true, y_proba)
roc_auc_value = auc(fpr, tpr)

# Plot
plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC =␣
↪{roc_auc_value:.3f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')  # random guess line
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC)')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```


Receiver Operating Characteristic (ROC)
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
ROC curve (AUC = 0.954)

```
[ ]:
```